

Python for AI: Core Programming & Logic



Eka Surya Aditya

AI/Machine Learning Engineer

Pendidikan: M.T (Master at Data Science/AI) - 2022

S.T (Teknik Informatika ITB) - 2020

Pengalaman:

Senior AI/ML Engineer at Telkomsel Ecosystem Digital (2025-Now)

Data Scientist at SVI (2023-2025)

AI Engineer at Pusat AI ITB (2020-2022)

OBJECTIVE & OUTLINE



Python for AI: Core Programming & Logic

Objektif: Memahami logika dasar Python sebagai bahasa pemrograman untuk AI Engineering.

Outline:

1. Python Fundamentals
2. Python Libraries for Data Processing
3. Working with API
4. Github

ReWork.

Python Fundamentals

Tipe data Python

Python types	Example expression
int	11, -14, 0, 2
float	21.3201, 0.0, 0.8, -2.34
str	"Hello Python 101"
boolean	True, False

```
In [4]: type(11)
```

```
Out[4]: int
```

```
In [5]: type(21.3201)
```

```
Out[5]: float
```

```
In [6]: type("Hello Python 101")
```

```
Out[6]: str
```

- Gunakan fungsi `type()` untuk mengetahui tipe dari sebuah expression.

Typecasting: Mengonversi Satu Tipe Data ke Tipe Data Lain

```
In [7]: # typecasting int to float  
float(2)
```

Out[7]: 2.0

```
In [8]: # typecasting float to int,  
# careful with the loss of precision  
int(1.1)
```

Out[8]: 1

```
In [9]: # typecasting str to int  
int('1')
```

Out[9]: 1

```
In [10]: int('A')
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-10-58e5992d18f7> in <module>  
----> 1 int('A')
```

ValueError: invalid literal for int() with base 10: 'A'

```
In [11]: # typecasting int to str  
str(1)
```

Out[11]: '1'

```
In [12]: # typecasting float to str  
str('2.2')
```

Out[12]: '2.2'

Variables

- Variables menyimpan dan memberikan nama pada nilai data.
- Variables tidak memiliki tipe bawaan; tipe hanya dimiliki oleh nilai data (int, float, str, bool, dan berbagai tipe lainnya).
 - Tipe variable saat ini mengikuti tipe nilai data yang sedang disimpan.
- Nilai data diberikan kepada variable menggunakan tanda '='.
 - Statement atau command tersebut disebut variable assignment.
 - Variable hanya menyimpan nilai data terbaru yang diberikan kepadanya.
- Setelah assignment, nilai data yang tersimpan dalam variable dapat digunakan dalam expression dengan menuliskan nama variable tersebut di dalam expression.



my_var

20

y

40

```
In [3]: 1 my_var = 1  
        2 my_var
```

Out[3]: 1

```
In [4]: 1 my_var = my_var+12  
        2 my_var
```

Out[4]: 13

```
In [5]: 1 my_var = 8  
        2 my_var
```

Out[5]: 8

```
In [6]: 1 my_var += 12  
        2 my_var
```

Out[6]: 20

```
1 y = my_var * 2  
2 y
```

40

Memeriksa Tipe Variable

- Kita dapat menggunakan command `type()` pada variable untuk memeriksa tipenya saat ini.

```
In [36]: my_var
```

```
Out[36]: 24
```

```
In [37]: type(my_var)
```

```
Out[37]: int
```

```
In [38]: y
```

```
Out[38]: 2.857142857142857
```

```
In [39]: type(y)
```

```
Out[39]: float
```

Penamaan Variable

- Sebaiknya gunakan nama yang bermakna untuk variable.
 - Nama variable diawali dengan huruf atau karakter underscore ('_'), kemudian dapat diikuti oleh sejumlah huruf atau angka.
- Misalkan kita memiliki data durasi tiga video: 42 menit, 57 menit, dan 43 menit. Berapa jam total durasi ketiga video tersebut?

```
In [41]: video1_duration_min = 42  
         video2_duration_min = 57  
         video3_duration_min = 43
```

```
In [42]: total_min = video1_duration_min + video2_duration_min + video3_duration_min  
         total_min
```

```
Out[42]: 142
```

```
In [43]: total_hr = total_min/60  
         total_hr
```

```
Out[43]: 2.3666666666666667
```

Lists

- List juga merupakan sequence yang berurutan, tetapi bersifat mutable, tidak seperti tuple atau string.
- List ditulis di dalam square brackets.
- List dapat dibuat secara nested atau memuat list lainnya.
 - List dan tuple dapat memuat data apa pun, termasuk list dan tuple lainnya.

```
In [32]: L = ['Surabaya', 3.4, 1293]
```

```
In [33]: L
```

```
Out[33]: ['Surabaya', 3.4, 1293]
```

```
In [34]: L1 = ['Surabaya', 3.4, 1293, [2, 5], ('A', 1)]
```

```
In [35]: L1
```

```
Out[35]: ['Surabaya', 3.4, 1293, [2, 5], ('A', 1)]
```

Lists: Mengakses Elements

- Elements pada list dapat diakses menggunakan index dengan cara yang sama seperti pada tuple atau string.

L = ['Surabaya', 3.4, 1293]

0	1	2
-3	-2	-1

In [36]: L[0]

Out[36]: 'Surabaya'

In [37]: L[1]

Out[37]: 3.4

In [38]: L[2]

Out[38]: 1293

In [39]: L[-3]

Out[39]: 'Surabaya'

In [40]: L[-2]

Out[40]: 3.4

In [41]: L[-1]

Out[41]: 1293

List Bersifat Mutable

- Tidak seperti tuple atau string, elements pada list dapat diubah.

```
In [42]:
```

```
L
```

```
Out[42]: ['Surabaya', 3.4, 1293]
```

```
In [43]:
```

```
L[0] = "Semarang"
```

```
In [44]:
```

```
L
```

```
Out[44]: ['Semarang', 3.4, 1293]
```

List Bersifat Mutable

- Jika satu list direferensikan oleh dua variables, perubahan elements melalui salah satu variable juga akan terlihat pada variable lainnya.

Suppose we have L and LL as follows:

```
In [52]: L = ['Surabaya', 3.4, 1293]
```

```
In [53]: LL = L
```

```
In [54]: L
```

```
Out[54]: ['Surabaya', 3.4, 1293]
```

```
In [55]: LL
```

```
Out[55]: ['Surabaya', 3.4, 1293]
```

We change L[1], and the change appear in LL too

```
In [56]: L[1] = 5.7
```

```
In [57]: L
```

```
Out[57]: ['Surabaya', 5.7, 1293]
```

```
In [58]: LL
```

```
Out[58]: ['Surabaya', 5.7, 1293]
```

List Slicing

- Slicing bekerja seperti pada tuple, tetapi menghasilkan list baru yang tidak berbagi reference dengan list lama.

```
In [63]: Lst = ['Surabaya', 3.4, 1293, "TR", 2]
```

```
In [64]: Lst35 = Lst[3:5]
```

Lst35 includes element at Lst[3]

```
In [65]: Lst35
```

```
Out[65]: ['TR', 2]
```

```
In [66]: Lst[3] = 1945
```

```
In [67]: Lst
```

Lst[3] is changed

```
Out[67]: ['Surabaya', 3.4, 1293, 1945, 2]
```

```
In [68]: Lst35
```

But, Lst35 is NOT changed!

```
Out[68]: ['TR', 2]
```

List Slicing

- Slicing dapat digunakan untuk menyalin sebuah list.

```
In [70]: Lst = ['Surabaya', 3.4, 1293, "TR", 2]
```

```
In [71]: Lst_c = Lst[:]
```

Lst_c is a copy of Lst

```
In [72]: Lst[0] = 'Bandung'
```

Lst[0] is changed

```
In [73]: Lst
```

```
Out[73]: ['Bandung', 3.4, 1293, 'TR', 2]
```

```
In [74]: Lst_c
```

But Lst_c is NOT changed!

```
Out[74]: ['Surabaya', 3.4, 1293, 'TR', 2]
```


List: Menambahkan Satu Element di Bagian Akhir

- `append()` hanya menambahkan satu element ke bagian akhir list.
- Jika argument-nya berupa list lain, list lama akan memuat list tersebut sebagai element terakhirnya.

```
In [95]: Lst = ['Surabaya', 3.4, 1293, "TR", 2]
```

```
In [96]: Lst
```

```
Out[96]: ['Surabaya', 3.4, 1293, 'TR', 2]
```

```
In [97]: Lst.append(5)
```

```
In [98]: Lst
```

```
Out[98]: ['Surabaya', 3.4, 1293, 'TR', 2, 5]
```

```
In [99]: Lst.append(['crocodile', 200])
```

```
In [100]: Lst
```

```
Out[100]: ['Surabaya', 3.4, 1293, 'TR', 2, 5, ['crocodile', 200]]
```

List: Menghapus Elements

- Kita dapat menghapus element pada index tertentu dari sebuah list menggunakan statement del.

```
In [100]: Lst
```

```
Out[100]: ['Surabaya', 3.4, 1293, 'TR', 2, 5, ['crocodile', 200]]
```

```
In [101]: del(Lst[6])
```

```
In [102]: Lst
```

```
Out[102]: ['Surabaya', 3.4, 1293, 'TR', 2, 5]
```

```
In [103]: del(Lst[2])
```

```
In [104]: Lst
```

```
Out[104]: ['Surabaya', 3.4, 'TR', 2, 5]
```

List Concatenation: Menggunakan Operator '+'

- Menggabungkan dua list akan menghasilkan list baru.

```
In [75]: Lst = ['Surabaya', 3.4, 1293, "TR", 2]
```

```
In [76]: Lst_plus = Lst + ['sharks', 100]
```

```
In [77]: Lst_plus
```

```
Out[77]: ['Surabaya', 3.4, 1293, 'TR', 2, 'sharks', 100]
```

```
In [78]: Lst[0] = 'Bandung'
```

```
In [79]: Lst
```

```
Out[79]: ['Bandung', 3.4, 1293, 'TR', 2]
```

```
In [80]: Lst_plus
```

```
Out[80]: ['Surabaya', 3.4, 1293, 'TR', 2, 'sharks', 100]
```

List

Cek isi list

```
a = 10
b = 2
list = [1, 2, 3, 4, 5]
print(a in list)
print(b in list)
```

False

True

```
a = 'a'
b = 'b'
list = ['h', 'a', 'l', 'o']
print(a in list)
print(b in list)
```

True

False

Tambah isi list

```
la = ["pepaya", "jeruk"]
la.append("semangka")
print(la)
print(len(la))
la[len(la):] = ["melon"]
print(la)
la[len(la):] = "apel"
print(la)
```

['pepaya', 'jeruk', 'semangka']

3

['pepaya', 'jeruk', 'semangka', 'melon']

['pepaya', 'jeruk', 'semangka', 'melon', 'a', 'p', 'e', 'l']

```
la = ["pepaya", "jeruk"]
la.extend(["semangka", "melon"])
print(la)
la[len(la):] = ["apel", "duku"]
print(la)
```

['pepaya', 'jeruk', 'semangka', 'melon']

['pepaya', 'jeruk', 'semangka', 'melon', 'apel', 'duku']

List: Memperpanjang List di Bagian Akhir

- Jika ingin memperpanjang list dengan elements dari list lain, gunakan method `extend()`.

```
In [90]: Lst = ['Surabaya', 3.4, 1293, "TR", 2]
```

```
In [91]: Lst
```

```
Out[91]: ['Surabaya', 3.4, 1293, 'TR', 2]
```

```
In [92]: Lst2 = ["crocodile", 200]
```

```
In [93]: Lst.extend(Lst2)
```

```
In [94]: Lst
```

```
Out[94]: ['Surabaya', 3.4, 1293, 'TR', 2, 'crocodile', 200]
```

- Lst diperpanjang dengan menambahkan sejumlah posisi di bagian akhir sesuai panjang Lst2.
- Elements dari Lst2 disalin secara berurutan ke posisi tambahan pada Lst1.

Dictionaries

- Dictionary merupakan kumpulan pasangan data.
 - Setiap pasangan terdiri dari key yang diikuti value dan dipisahkan oleh tanda titik dua.
- Keys dalam dictionary membentuk sebuah set:
 - Setiap key bersifat unik dan immutable.
 - Posisi keys dalam dictionary tidak dicatat.
- Values dapat bersifat immutable atau mutable dan dapat memiliki duplikat.
- Dictionary ditandai dengan sepasang curly brackets; setiap pasangan key-value dipisahkan dari pasangan lainnya menggunakan tanda koma.

Dictionary: Analogi dengan List

- Dictionary serupa dengan list, tetapi menggunakan arbitrary immutable objects sebagai index, bukan integers.

List

Index	Element
0	Element1
1	Element2
2	Element3
3	Element4
...	...

Dictionary

Key: is an index by label

Value

Key 1	Value 1
Key 2	Value 2
Key 3	Value 3
Key 4	Value 4
...	...

Dictionary

```
In [37]: city_pop = {"Jakarta": 10.1, "Surabaya": 3.4, "Bogor": 1.1, "Depok": 1.8, \
                    "Bandung": 2.5, "Medan": 2.2, "Makassar": 1.4, "Denpasar": 0.9}
```

```
In [38]: city_pop
```

```
Out[38]: {'Jakarta': 10.1,
          'Surabaya': 3.4,
          'Bogor': 1.1,
          'Depok': 1.8,
          'Bandung': 2.5,
          'Medan': 2.2,
          'Makassar': 1.4,
          'Denpasar': 0.9}
```

"Jakarta"	10.1
"Surabaya"	3.4
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9

Dictionary: Mengakses Elements Menggunakan Key

```
city_pop = {"Jakarta": 10.1, "Surabaya": 3.4, "Bogor": 1.1, "Depok": 1.8,\n            "Bandung": 2.5, "Medan": 2.2, "Makassar": 1.4, "Denpasar": 0.9}
```

"Jakarta"	10.1
"Surabaya"	3.4
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9

```
In [39]: city_pop["Jakarta"]
```

```
Out[39]: 10.1
```

```
In [40]: city_pop["Surabaya"]
```

```
Out[40]: 3.4
```

```
In [41]: city_pop['Bogor']
```

```
Out[41]: 1.1
```

```
In [42]: city_pop['Denpasar']
```

```
Out[42]: 0.9
```

Dictionary: Menambahkan Elements

"Jakarta"	10.1
"Surabaya"	3.4
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9

In [43]: `city_pop['Ambon'] = 0.4`



"Jakarta"	10.1
"Surabaya"	3.4
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9
"Ambon"	0.4

Dictionary: Menambahkan Elements

"Jakarta"	10.1
"Surabaya"	3.4
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9

In [43]: `city_pop['Ambon'] = 0.4`



In [44]: `city_pop`

Out[44]: {'Jakarta': 10.1,
'Surabaya': 3.4,
'Bogor': 1.1,
'Depok': 1.8,
'Bandung': 2.5,
'Medan': 2.2,
'Makassar': 1.4,
'Denpasar': 0.9,
'Ambon': 0.4}

Dictionary: Menghapus Elements

"Jakarta"	10.1
"Surabaya"	3.4
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9
"Ambon"	0.4

In [45]: `del(city_pop['Surabaya'])`



"Jakarta"	10.1
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9
"Ambon"	0.4

Dictionary: Menghapus Elements

"Jakarta"	10.1
"Surabaya"	3.4
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9
"Ambon"	0.4

In [45]: `del(city_pop['Surabaya'])`

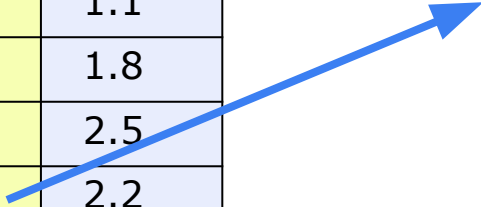


In [46]: `city_pop`

Out[46]: {'Jakarta': 10.1,
'Bogor': 1.1,
'Depok': 1.8,
'Bandung': 2.5,
'Medan': 2.2,
'Makassar': 1.4,
'Denpasar': 0.9,
'Ambon': 0.4}

Dictionary: Memeriksa Apakah Element Terdapat di Dalam Dictionary

"Jakarta"	10.1
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9
"Ambon"	0.4



```
In [47]: "Medan" in city_pop
```

```
Out[47]: True
```

```
In [48]: "Surabaya" in city_pop
```

```
Out[48]: False
```

Dictionary: Mendapatkan Semua Keys Menggunakan Method keys()

"Jakarta"	10.1
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9
"Ambon"	0.4

- keys() menghasilkan list-like object yang memuat keys dari dictionary dan dapat dikonversi menjadi list atau collection lainnya.

```
In [49]: city_pop.keys()
```

```
Out[49]: dict_keys(['Jakarta', 'Bogor', 'Depok', 'Bandung', 'Medan', 'Makassar', 'Denpasar', 'Ambon'])
```

Dictionary: Mendapatkan Semua Keys Menggunakan Method keys()

"Jakarta"	10.1
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9
"Ambon"	0.4

```
In [50]: list(city_pop.keys())
```

```
Out[50]: ['Jakarta',  
          'Bogor',  
          'Depok',  
          'Bandung',  
          'Medan',  
          'Makassar',  
          'Denpasar',  
          'Ambon']
```


Dictionary: Mendapatkan Semua Values Menggunakan Method values()

"Jakarta"	10.1
"Bogor"	1.1
"Depok"	1.8
"Bandung"	2.5
"Medan"	2.2
"Makassar"	1.4
"Denpasar"	0.9
"Ambon"	0.4

- Method values() bekerja dengan cara serupa, tetapi digunakan untuk mendapatkan semua values dalam dictionary.

```
In [51]: list(city_pop.values())
```

```
Out[51]: [10.1, 1.1, 1.8, 2.5, 2.2, 1.4, 0.9, 0.4]
```

Statement if

- Misalkan akses masuk ke sebuah tempat wisata hanya diberikan kepada pengunjung yang berusia maksimal 12 tahun.
 - Contohnya, jika usia pengunjung 13, 14, atau lebih, akses tidak diberikan dan pengunjung melanjutkan aktivitas lainnya.
 - Jika usia pengunjung 12, 11, atau kurang, akses diberikan dan pengunjung melanjutkan aktivitas setelah menikmati wahana.
- Branching tersebut dapat ditulis menggunakan statement if.

if CONDITION:

do_something_only_when_condition_holds

everyone_do_something

- Setelah if-condition, statements yang hanya ingin dijalankan ketika if-condition bernilai true WAJIB ditulis menggunakan indentation!

The if Statement

In [21]: `age = 11`

In [22]: `if age <= 12:
 print("You may enter.")
 print("Move on")`

You may enter.
Move on

Karena 11 lebih kecil dari atau sama dengan 12, if-condition terpenuhi.

Indented statements dijalankan ketika if-condition bernilai true.

Statement ini dijalankan terlepas dari apakah if-condition bernilai true atau false.

In [23]: `age = 13`

In [24]: `if age <= 12:
 print("You may enter.")
 print("Move on")`

Move on

Karena 13 lebih besar dari 12, indented statement TIDAK dijalankan.

Statement if-else: Memilih Salah Satu dari Dua Alternatif

```
In [4]: age = 11
if age <= 12:
    print("You may enter attraction A")
else:
    print("You may enter attraction B")
print("Move on")
```

You may enter attraction A
Move on

If-condition bernilai true: 11 lebih kecil dari atau sama dengan 12.

Bagian ini dijalankan.

Bagian ini TIDAK dijalankan.

Bagian ini dijalankan terlepas dari nilai if-condition.

```
In [5]: age = 13
if age <= 12:
    print("You may enter attraction A")
else:
    print("You may enter attraction B")
print("Move on")
```

You may enter attraction B
Move on

If-condition bernilai false: 13 lebih besar dari 12.

Bagian ini TIDAK dijalankan.

Bagian ini dijalankan.

Bagian ini dijalankan terlepas dari nilai if-condition.

The else
statement

Statement elif: Salah Satu dari Beberapa Alternatif

The elif statement

Condition bernilai true: 11 lebih kecil dari atau sama dengan 12.

```
age = 11
if age <= 12:
    print("You may enter attraction A")
elif age == 13:
    print("You may enter special attraction X")
else:
    print("You may enter attraction B")
print("Move on")
```

Dijalankan.

TIDAK dijalankan.

```
You may enter attraction A
Move on
```

Bagian ini selalu dijalankan.

Statement elif: Salah Satu dari Beberapa Alternatif

The elif statement

```
age = 13
if age <= 12:
    print("You may enter attraction A")
elif age == 13:
    print("You may enter special attraction X")
else:
    print("You may enter attraction B")
print("Move on")
```

If-condition bernilai false: 13 lebih besar dari 12.

TIDAK dijalankan

Elif-condition bernilai true:
13 sama dengan 13.

Dijalankan

Bagian ini selalu
dijalankan

You may enter special attraction X
Move on

Statement elif: Salah Satu dari Beberapa Alternatif

The elif statement

```
age = 15
if age <= 12:
    print("You may enter attraction A")
elif age == 13:
    print("You may enter special attraction X")
else:
    print("You may enter attraction B")
print("Move on")
```

If-condition bernilai false: 13 lebih besar dari 12.

TIDAK dijalankan.

Elif-condition bernilai false: 15 tidak sama dengan 13.

Dijalankan.

Bagian ini selalu dijalankan.

You may enter attraction B
Move on

The if, else, and elif statements

- Secara umum, bentuk yang dapat digunakan terdiri dari:
 - Sebuah statement if
 - Diikuti oleh nol atau lebih statements elif
 - Kemudian nol atau satu statement else.
- Statement if dan seluruh statements elif memiliki Boolean condition.
 - Boolean conditions diperiksa secara berurutan dari atas ke bawah hingga ditemukan condition yang bernilai true.
 - Setelah condition yang bernilai true ditemukan, indented statements di bawah statement if/elif tersebut dijalankan. Setelah itu, program keluar dari keseluruhan block if-elif-else.
 - Jika tidak ada condition yang bernilai true dan terdapat statement else, indented statements di bawahnya akan dijalankan. Setelah itu, program keluar dari keseluruhan block if-elif-else. Jika tidak terdapat statement else, program langsung keluar dari block if-elif-else.

The range object constructor

1. `range(stop)`
 - `stop` harus berupa positive integer; jika tidak, object range akan kosong.
 - Menghasilkan integers `[0, 1, ..., stop-1]` dengan `length = stop`.
2. `range(start, stop)`
 - `start` dan `stop` harus berupa integers
 - Nilai `start < stop`; jika tidak, object range akan kosong
 - Menghasilkan integers `[start, start + 1, ..., stop]` □ `length = stop - start`
3. `range(start, stop, step)`
 - `start` and `stop` must be integers and `step` must be nonzero integer
 - if `step > 0`, then `start < stop` must hold, otherwise the range object is empty
 - if `step < 0`, then `start > stop` must hold, otherwise the range object is empty
 - generates integers `[start, start + step, start + 2*step, ..., start + k*step]` where `k` is the largest integer not exceeding $(|stop - start|)/step$.

The range object

```
list(range(5))
```

```
[0, 1, 2, 3, 4]
```

```
list(range(3,12))
```

```
[3, 4, 5, 6, 7, 8, 9, 10, 11]
```

```
list(range(-4,5))
```

```
[-4, -3, -2, -1, 0, 1, 2, 3, 4]
```

```
list(range(3,12,2))
```

```
[3, 5, 7, 9, 11]
```

```
list(range(-4,5,3))
```

```
[-4, -1, 2]
```

```
list(range(10,3,-1))
```

```
[10, 9, 8, 7, 6, 5, 4]
```

```
list(range(6, -5, -2))
```

```
[6, 4, 2, 0, -2, -4]
```

Loops: for statements

- Misalkan tersedia sebuah list yang berisi nama-nama warna, misalnya

`["red", "orange", "green", "purple", "blue"]`

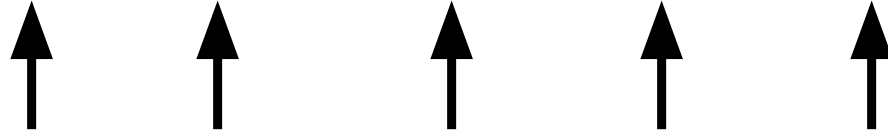
- Kita ingin mengganti setiap nama warna menjadi "black" sehingga hasilnya

`["black", "black", "black", "black", "black"]`

- Bagaimana cara melakukannya?

0	1	2	3	4
---	---	---	---	---

colors = ["red", "orange", "green", "purple", "blue"]



for colors 0 in colors, colors 0 =
black
for colors 1 in colors, colors 1 =
black
for colors 2 in colors, colors 2 =
black
for colors 3 in colors, colors 3 =
black
for colors 4 in colors, colors 4 =
black



- running index from 0 to 4
- ```
for i in range(0,5):
 colors[i] = "black"
```
- range(0,5) returns the sequence [0, 1, 2, 3, 4]

```
1 colors = ["red", "orange", "green", "purple", "blue"]
2
3 for i in range(0,5):
4 colors[i] = "black"
5
6 print(colors)
```

['black', 'black', 'black', 'black', 'black']

# Loop: for statement

- Jika kita hanya ingin menggunakan value dari setiap element dalam list tanpa mengubah list tersebut, kita dapat melakukan iteration langsung pada elements tanpa menggunakan index.

```
colors = ["red", "orange", "green", "purple", "blue"]

for color in colors:
 print(color.upper())
```

```
RED
ORANGE
GREEN
PURPLE
BLUE
```

Lakukan iteration langsung pada elements dalam list.

color akan mengambil value "red", kemudian "orange", lalu "green", dan seterusnya.

# Contoh For Lainnya

```
: for x in "banana":
 print(x)
```

b  
a  
n  
a  
n  
a

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
 print(x)
 if x == "banana":
 break
```

apple  
banana

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
 if x == "banana":
 break
 print(x)
```

apple

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
 if x == "banana":
 continue
 print(x)
```

apple  
cherry

# Nested Loop

```
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
 for y in fruits:
 print(x, y)
```

```
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
```

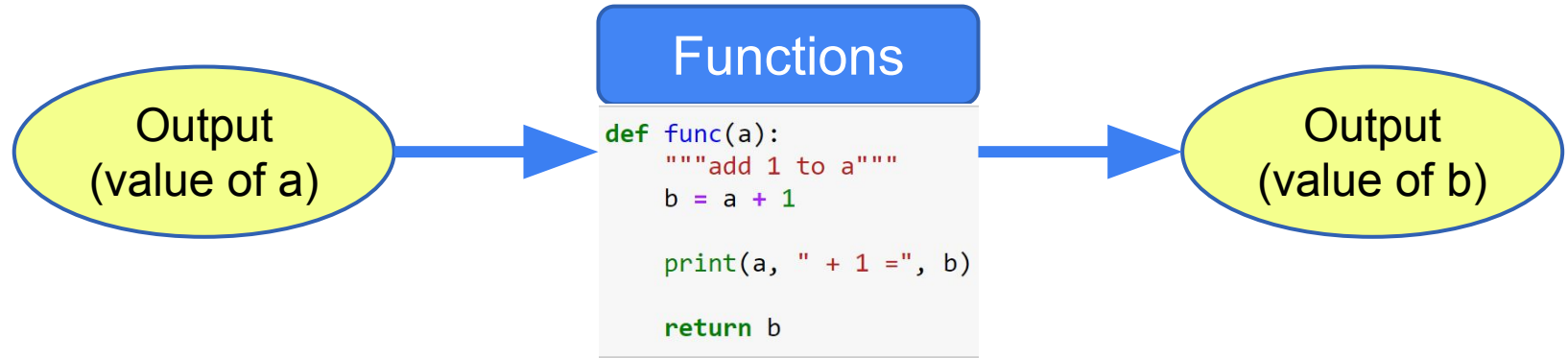
```
for x in range(1,5):
 str=""
 for i in range(5,x,-1):
 str += " "
 for i in range(0,x*2-1):
 str += "*"
 print(str)
```

```
*


```



# Functions



- Functions menerima sejumlah input dan menghasilkan output atau perubahan tertentu.
- Function merupakan bagian kode yang dapat digunakan kembali.
- Kita dapat mendefinisikan function sendiri.
- Namun, lebih sering kita menggunakan functions yang dibuat oleh orang lain.
  - Kita hanya perlu memahami cara kerja function tersebut, termasuk input dan output-nya.
  - Terkadang, kita juga perlu mengetahui cara mengimpor function ke dalam program.

```
x = 30
print(x)
```

```
if x % 2 == 0:
 y = x // 2
else:
 y = 3 * x + 1
```

```
z = y
print(z)
```

```
if z % 2 == 0:
 y = z // 2
else:
 y = 3 * z + 1
```

```
u = y
print(u)
```

```
if u % 2 == 0:
 y = u // 2
else:
 y = 3 * u + 1
```

```
w = y
print(w)
```

## Shorter main code

```
x = 30
print(x)

z = func(x)
print(z)

u = func(z)
print(u)

w = func(u)
print(w)
```

Bagian-bagian kode yang serupa dan berulang dipisahkan menjadi sebuah function. Main code mengirimkan sebuah value kepada function dan menerima return value dari function tersebut.

```
def func(a):
 """return a/2 if a is even
 and 3a + 1 if a is odd"""
 if a % 2 == 0:
 b = a // 2
 else:
 b = 3 * a + 1

 return b
```

# Membuat Functions

Gunakan nama yang deskriptif.

- Gunakan add1 karena kita ingin menghasilkan input ditambah 1.

Start with the keyword  
**def**

Input function diberikan sebagai variables yang disebut formal parameters dan ditulis di dalam parentheses.

Documentation string. Pemanggilan `help(add1)` akan menampilkan string ini.

Indented code block yang berisi statements.

Hasil function dikembalikan menggunakan keyword **return**.

Setelah function didefinisikan, kita dapat memanggilnya.

```
def add1(a):
 """Return a + 1"""
 b = a + 1
 return b

add1(7)
```

# Statement return Bersifat Opsional

```
def jakarta():
 print("DKI Jakarta")

jakarta()
```

DKI Jakarta

```
def no_work():
 pass

print(no_work())
```

None

- Dalam eksekusi function, Python akan keluar dari function
  - Ketika statement return dijalankan atau
  - Tidak ada lagi statement yang perlu dijalankan.
- Oleh karena itu, function dapat dibuat tanpa statement return
  - Misalnya ketika kita hanya ingin mencetak sesuatu atau melakukan perubahan pada object/data.
- Jika function keluar BUKAN melalui statement return, Python secara otomatis akan mengembalikan object bertipe None kepada caller.
- Function body tidak boleh kosong.
  - Function sederhana yang mengembalikan None dapat menggunakan statement pass sebagai satu-satunya statement.
  - **Pass** berarti 'tidak melakukan apa pun'.

# Function Dapat Menjalankan Beberapa Tugas

```
def add2(a):
 b = a + 2
 print(a, "plus 2 equals", b)
 return b
```

```
add2(5)
```

5 plus 2 equals 7

7

- Function ini:
  - Mencetak sebuah statement
  - Mengembalikan argument yang telah ditambah 2.

**Any question guys ~**

ReWork.

# Python Libraries for Data Processing

# NumPy

- Library dasar untuk perhitungan saintifik (*scientific computing*) dengan Python (<https://numpy.org/>)
- Alternatif untuk Python List: Numpy Array untuk  $n$ -dimensi
- Mudah digunakan dan bersifat *open source*
- Jika library belum terpasang, tuliskan perintah instalasi:

```
pip install numpy
```

- Kemudian impor:

```
import numpy as np
```

```
In [6]: import numpy as np
```

```
In [7]: np_height = np.array(height)
```

```
In [8]: np_height
```

```
Out[8]: array([1.84, 1.79, 1.82, 1.9, 1.8])
```

```
In [9]: np_weight = np.array(weight)
```

```
In [10]: np_weight
```

```
Out[10]: array([66.5, 60.3, 64.7, 89.5, 69.8])
```

```
In [11]: bmi = np_weight / np_height ** 2
```

```
In [12]: bmi
```

```
Out[12]: array([19.64201323, 18.81963734, 19.53266514, 24.79224377, 21.54320988])
```



# NumPy

- NumPy juga dapat digunakan untuk membuat array berdimensi- $n$

```
In [13]: import numpy as np
```

```
In [14]: np_height = np.array([1.84, 1.79, 1.82,
1.9, 1.8])
```

```
In [15]: np_weight = np.array([66.5, 60.3, 64.7,
89.5, 69.8])
```

```
In [16]: type(np_height)
Out[16]: numpy.ndarray
```

```
In [16]: type(np_weight)
Out[16]: numpy.ndarray
```

*ndarray = n-dimensional array*

```
In [17]: np_2d = np.array([[1, 2, 3, 4, 5],
[6, 7, 8, 9, 10]])
```

```
In [18]: np_2d
Out[18]: array([[1, 2, 3, 4, 5],
[6, 7, 8, 9, 10]])
```

```
In [19]: np_2d.shape
Out[19]: (2, 5)
```

Array berdimensi 2 baris 5 kolom  $\rightarrow$  Matriks  $M_{2 \times 5}$

$$M = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \end{bmatrix}$$

# NumPy

- NumPy dapat digunakan untuk membuat array artifisial dengan method `arange()` dan `linspace()`

```
arr_even = np.arange(0,101,2)
arr_even
```

```
array([0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24,
 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50,
 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76,
 78, 80, 82, 84, 86, 88, 90, 92, 94, 96, 98, 100])
```

```
arr_5 = np.linspace(0,100,21)
arr_5
```

```
array([0., 5., 10., 15., 20., 25., 30., 35., 40., 45., 50.,
 55., 60., 65., 70., 75., 80., 85., 90., 95., 100.])
```



- Pandas (Panel Data) merupakan library populer di Python yang digunakan untuk *data structure* dan *data analysis*
- Bersifat *open source* dan tersedia di <https://pandas.pydata.org/>
- Pandas sangat berkaitan dengan NumPy
- Jika library belum terpasang, tuliskan perintah instalasi:  

```
pip install pandas
```
- Kemudian impor:  

```
import pandas as pd
```

### ***Data Wrangling / Data Munging***

- *Reshaping* (mengubah bentuk data)
- *Joining* (menggabungkan data)
- *Splitting* (pemisahan data)
- *Time-series analysis* (data berkala)

### ***Data Cleansing***

- Membersihkan data tidak lengkap (*Error*)
- Menangani data pencilan (*outliers*)
- Menghapus data duplikat

# Representasi Data di pandas

- Terdapat 2 *data objects*: *Series* dan *DataFrame*
- *Series* □ Data berbentuk 1 dimensi

```
In [13]: np.array([1, 2, 3, 4, 5])
```

```
Out[13]: array([1, 2, 3, 4, 5])
```

- *DataFrame* □ Data berbentuk 2 dimensi atau lebih

```
In [14]: np.array([[1, 2], [3, 4]])
```

```
Out[14]: array([[1, 2],
 [3, 4]])
```

Kolom: Fitur / atribut

|    | Negara    | Populasi | Area   | Ibukota   |
|----|-----------|----------|--------|-----------|
| IN | Indonesia | 250      | 123456 | Jakarta   |
| MA | Malaysia  | 25       | 3456   | KL        |
| SI | Singapura | 15       | 456    | Singapura |
| JP | Jepang    | 60       | 5678   | Tokyo     |
| TH | Thailand  | 45       | 678    | Bangkok   |

Baris: sampel



- Pandas dapat mengimpor data dari berbagai format: *comma-separated value* (CSV), file teks, Microsoft Excel, database SQL, dan format HDF5
- Unduh dataset: <http://bit.ly/TabDataset>
- CSV file □ DataFrame

```
import pandas as pd
```

```
,M,Populasi,Area,Ibukota
IN,Indonesia,250,123456,Jakarta
MA,Malaysia,25,3456,KL
SI,Singapura,15,456,Singapura
JP,Jepang,60,5678,Tokyo
TH,Thailand,45,678,Bangkok
```

```
In [1]: Tab = ... # deklarasi tabel
```

```
In [2]: Tab
```

|    | Negara    | Populasi | Area   | Ibukota   |
|----|-----------|----------|--------|-----------|
| IN | Indonesia | 250      | 123456 | Jakarta   |
| MA | Malaysia  | 25       | 3456   | KL        |
| SI | Singapura | 15       | 456    | Singapura |
| JP | Jepang    | 60       | 5678   | Tokyo     |
| TH | Thailand  | 45       | 678    | Bangkok   |



```
In [3]: import pandas as pd
```

```
In [4]: Tab = pd.read_csv("Tab.csv")
```

```
In [5]: Tab
```

```
Out[5]:
```

|   | Unnamed: 0 | Negara    | Populasi | Area   | Ibukota   |
|---|------------|-----------|----------|--------|-----------|
| 0 | IN         | Indonesia | 250      | 123456 | Jakarta   |
| 1 | MA         | Malaysia  | 25       | 3456   | KL        |
| 2 | SI         | Singapura | 15       | 456    | Singapura |
| 3 | JP         | Jepang    | 60       | 5678   | Tokyo     |
| 4 | TH         | Thailand  | 45       | 678    | Bangkok   |

```
In [6]: Tab["Negara"] # akses kolom
```

```
Out[6]:
```

```
0 Indonesia
1 Malaysia
2 Singapura
3 Jepang
4 Thailand
Name: Negara, dtype: object
```

```
In [7]: Tab.Ibukota # akses kolom
```

```
Out[7]:
```

```
0 Jakarta
1 KL
2 Singapura
3 Tokyo
4 Bangkok
Name: Ibukota, dtype: object
```

ReWork.

# Hands On

ReWork.

# Working with API



# Apa itu API?

**API adalah "perantara"** yang memungkinkan satu aplikasi meminta data atau layanan dari aplikasi atau sistem lain. Kalian tidak perlu tahu detail implementasi di baliknya — cukup tahu cara memintanya.



**Analogi Restoran:** Kalian tidak perlu tahu bagaimana chef memasak. Cukup pesan melalui pelayan (API), dan makanan datang ke meja kalian.

Dengan API, aplikasi bisa saling berkomunikasi dengan mudah dan aman — tanpa perlu mengintip "dapur" satu sama lain.

## Format Data JSON

### Mengapa JSON?

**JSON adalah format standar** untuk pertukaran data di hampir seluruh API modern. Strukturnya mudah dibaca manusia sekaligus mudah diproses mesin.

- Gunakan `{}` untuk *object* (satu entitas)
- Gunakan `[]` untuk *array* (kumpulan entitas)
- Setiap data memiliki **key** (nama field) dan **value** (isinya)

### Contoh JSON

```
{
 "nama": "Budi",
 "umur": 25,
 "kota": "Jakarta"
}
```

```
{
 "users": [
 { "id": 1, "nama": "Andi" },
 { "id": 2, "nama": "Siti" }
]
}
```

# HTTP Request & Response



## Komponen Request

### Method

GET · POST · PUT · DELETE

### Endpoint/URL

Alamat tujuan permintaan

### Headers

Tipe data, autentikasi, dll

## Komponen Response

### Status Code

200 Berhasil · 404 Tidak ditemukan · 500 Error server

### Body

Data yang dikembalikan server, biasanya dalam format JSON

# Format Data JSON

## Mengapa JSON?

JSON adalah format standar untuk pertukaran data di hampir seluruh API modern. Strukturnya mudah dibaca manusia sekaligus mudah diproses mesin.

- Gunakan {} untuk *object* (satu entitas)
- Gunakan [] untuk *array* (kumpulan entitas)
- Setiap data memiliki **key** (nama field) dan **value** (isinya)

## Contoh JSON

```
{
 "nama": "Budi",
 "umur": 25,
 "kota": "Jakarta"
}
```

```
{
 "users": [
 { "id": 1, "nama": "Andi"
 },
 { "id": 2, "nama": "Siti" }
]
}
```

# Dari Visual ke Kode Python

Sekarang kita lihat bagaimana melakukan request yang sama di Postman atau Network Tab, tapi menggunakan **Python dengan library Requests**.

## Contoh Kode

```
import requests

response = requests.get(
 "https://api.example.com/users"
)

Baca response sebagai JSON
data = response.json()

print(data)
```

## Apa yang Terjadi?

### 1 Import library

Aktifkan Requests agar Python bisa melakukan HTTP request

### 2 Kirim GET request

Python mengirim permintaan ke endpoint yang dituju, persis seperti Postman

### 3 Baca JSON

`.json()` mengubah response menjadi data Python yang siap dipakai



Apa yang tampil di Postman atau Network Tab adalah **hasil yang sama** dengan apa yang dicetak Python – kalian sudah menghubungkan visual dengan kode!

**Any question guys ~**

ReWork.

**Github**



# Version Control System: Mengapa Penting untuk Developer

Hari ini kita akan belajar tentang Git dan GitHub—tools yang akan **mengubah cara** kalian mengelola kode.



## Riwayat Lengkap

VCS mencatat **setiap perubahan kode** dari waktu ke waktu—tidak ada yang terlewat.



## Bandingkan & Kembali

Lihat apa yang berubah, bandingkan versi, dan **kembalikan ke versi sebelumnya** jika ada kesalahan.

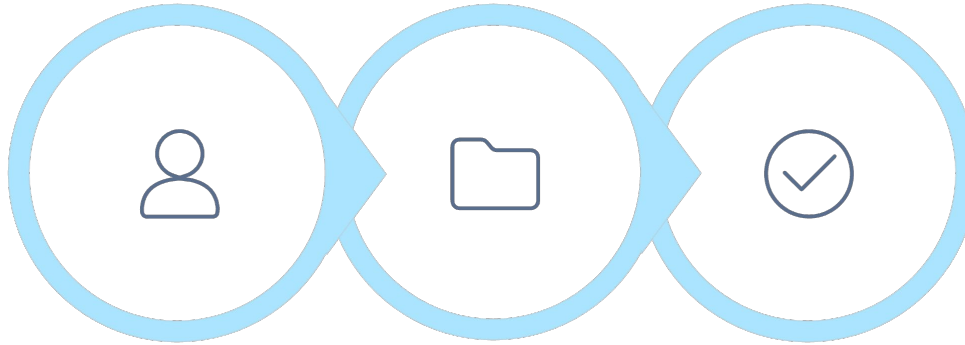


## Kolaborasi Tim

Fondasi kerja tim yang efisien—**tanpa VCS, chaos menanti!**

# Konfigurasi Awal & Inisialisasi Repository

Sekarang kita akan melakukan setup pertama kali. Saya akan menunjukkan langkah demi langkah di terminal.



**git config**

**git init**

**Siap Melacak**

## Konfigurasi Identitas

```
git config --global user.name "Nama Kamu"
git config --global user.email "email@kamu.com"
```

## Inisialisasi Repository

```
cd nama-folder-project
git init
```

Setelah ini, folder Anda siap untuk melacak perubahan kode!



# Tracking, Commit, Push & Pull: Workflow Seharian

Ini adalah workflow yang akan kalian gunakan **setiap hari**. Mari kita praktik bersama-sama!

1

## Cek Status

```
git status
```

Lihat file mana yang berubah dan belum di-track.

2

## Staging Area

```
git add .
```

Masukkan perubahan ke staging area sebelum commit.

3

## Commit

```
git commit -m "pesan deskriptif"
```

Pesan yang baik membantu tim memahami apa yang Anda ubah.

4

## Hubungkan Remote

```
git remote add origin [URL]
```

Hubungkan repository lokal ke GitHub.

5

## Push & Pull

```
git push
git pull
```

Kirim & ambil perubahan dari GitHub—siap berkolaborasi!

📌 Dengan menguasai workflow ini, kalian sudah bisa berkolaborasi dengan tim secara profesional! 🚀

ReWork.

**Thank You** 🙌